

Personal Media Tracker

SYSTEM DESIGN SPECIFICATION

Eric Chen, Sasha Crawford, and Jessamine Magalona
DESALES UNIVERSITY | 2755 STATION AVE. CENTER VALLEY PA

Software Design Document Specification Template

The Software Design Specification (SDS) sections provide you with guidelines related to the structure and the contents of SDS document. The Software Design Specification document includes at least these sections.

For the project, your team may have good reasons for wanting to deviate from this proposed outline. If a section is not applicable in your case, do not delete it; instead, give the topic heading and write "Not applicable".

You will note that there is some overlap in the content between different documents (i.e. the User Requirements Specification Document and the Software Design Specification Document.) This redundancy allows each document to stand on its own.

**ONLY THE SECTION TITLES COLORED IN ORANGE
ARE REQUIRED TO BE COMPLETED.**

**DO NOT DELETE THE SECTIONS YOU ARE NOT
COMPLETING AS THEY ARE A PART OF THE
DOCUMENT**

Contents

1	Introduction	3
1.1	Purpose of this document	3
1.2	Scope of the development project	3
1.3	Definitions, acronyms, and abbreviations	3
1.4	References	3
1.5	Overview of document	3
2	System architecture description	3
2.1	Overview of modules / components	3
2.2	Structure and relationships	3
2.3	User interface	3
2.4	User interface issues	3
3	Detailed description of components (At least 6)	4
3.1	Search Component	4
3.2	Sort Component	5
3.3	Tracking Component	6
3.4	Profile Component	6
4	Reuse and relationships to other products	7
5	Design decisions and tradeoffs	7
6	Pseudocode for components	7
7	Appendices (if any)	7
	Software component template for section 3	7

1 INTRODUCTION

1.1 PURPOSE OF THIS DOCUMENT

Full description of the main objectives of the SDS document.

1.2 SCOPE OF THE DEVELOPMENT PROJECT

This will be similar to what was written in the SRS.

1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

Be sure to alphabetize!

1.4 REFERENCES

This section will include technical books and documents related to design issues. Be certain that the references you give are complete and in the appropriate format.

1.5 OVERVIEW OF DOCUMENT

A short description of how the rest of the SDS is organized and what can be found in the rest of the document. This is not simply a table of contents. Motivate and briefly describe the various parts!

2 SYSTEM ARCHITECTURE DESCRIPTION

2.1 OVERVIEW OF MODULES / COMPONENTS

The Personal Media Tracker includes a variety of several key components. The search module allows for the users to retrieve media titles from an external database. The tracking module permits the user to add and manage titles as they choose. The profile module manages user data and associated data with the active account. The review module organizes ratings and written reviews. The sort and filter module organizes displayed media results from the search module. The data access module retrieves user-specific data, and the watch status module permits users to update viewing progress. These components work together to perform the functionality of the media tracking system.

2.2 STRUCTURE AND RELATIONSHIPS

The architecture of the system is modular. The user interacts with the application through a user interface. The user interface then communicates with the processing components, such as search and sort. The search module interacts with an external database to retrieve various media information such as titles, whereas the tracking and profile modules interact with a local storage database to save user specific data.

2.3 USER INTERFACE

The user interface includes multiple screens for the application. This includes the search interface, profile selection, and media tracking. The user can utilize the search interface for locating media titles and selecting which titles to track. A tracking form allows users to input their viewing progress as well as personal ratings and reviews. The interface also includes sorting and filtering for organizing search results. Error messaging is included for no matching titles found or any invalid inputs when necessary.

2.4 USER INTERFACE ISSUES

The user interface is designed to be simple and intuitive for the user. The navigation is simplified and consistent for user efficiency. Features such as tracking, searching, and sorting as easily accessible. Error messages are designed to

inform the user where invalid inputs are and guide the user to correct inputs. The interface focuses on simplicity to enhance usability and efficiency.

3 DETAILED DESCRIPTION OF COMPONENTS (AT LEAST 6)

3.1 SEARCH COMPONENT

<u>Identification</u>	Search
<u>Type</u>	Module
<u>Purpose</u>	Search must allow the user to look for with the information stored on various databases by typing in the designated text field on Personal Media Tracker. Given a title or keyword, search must be able to provide a matching title or display a message showing no matches in 2 seconds.
<u>Function</u>	Search receives data from a text field on the user interface. It processes this input by going through its connected database of media titles, searching a match to the user-entered data. If matching is successful, the system will collect information about title and return it to the user interface. If no matches found, the following message will be returned to the user interface: "No matches found".
Subordinates	Search is composed of various constituents. A connector between the user and the database allows for processing of user input and access to information in the database. Regarding user input, another module receives the request while another module will search the database for titles matching the user input. Finally, a response module will return the results of the previous module search, either related titles or an error message to the user interface.
<u>Dependencies</u>	Search depends on other system components to perform its functions effectively. It relies on the user interface component to receive user information to search its database for. The component also depends on the database as it requires titles to match user input. These components interact in a sequence where user data is first collected through the interface, looked through, and finally returned related titles or an error message.
<u>Interfaces</u>	Search communicates with external and internal interfaces to receive data and return results. The primary external interface is the text field where users enter titles or keywords. Internally, the component communicates with backend services and the database using structured parameters and data requests. The system provides clear feedback through messages that indicate a successful search by returning the requested titles or error message stating, "No matches found".
Resources	Search requires a software requirement for DBMS. The search module will be applicable for various functions, including the search through every title available in large database and smaller database associated with the user's account of logged media titles.
Processing	Search begins when the user enters a keyword or title into the designated text field and sends it. When the system receives the user input, the system prepares the input by standardizing the text, such as removing unnecessary spaces or converting it to a consistent format to ensure accurate comparison. Then, it accesses the related database and compares the user-entered text with the titles stored in it. While in the database, it checks each record to determine whether the entered text matches any existing media title. If a match is found, the system gathers the relevant information associated with that title and stores it as part of the search results. After all records have been checked, if one or more matches exist, the collected information is returned to the user interface and displayed to the user. If no matching titles are found after the search is completed, the system returns the message "No matches found."
Data	The data representation of the Search module will be primarily text-based, matching the media titles in the related database. There will no initial value held in the Search module as user input will not be stored long term when compared saving a user's watch status of a media title.

3.2 SORT COMPONENT

<u>Identification</u>	Sort
<u>Type</u>	Module
<u>Purpose</u>	Sort allows the user to categorize media titles from various databases on Personal Media Tracker, making it easier to find what they want. This component when implemented fits the media titles into user-specified criteria, improving usability and readability in the application UI. Sort must be able to handle various list sizes. Additionally, the component must maintain consistent ordering and efficient system response times.
<u>Functions</u>	Sort receives a list of media records and various options of sorting criteria from another component in the system. Each record contains various attributes about a media title. These attributes inform how the media titles can be filtered when specified in a dropdown menu. In the dropdown menu, the specified attributes appear. The user will select one of the categories. A sorting algorithm will take place, rearranging them based on the specified attribute.
<u>Subordinates</u>	The Sort component consists of several internal elements that support its functionality. A controller portion of the component initiates the sorting process and determines the sorting criteria provided by the calling component. A comparison function evaluates two records at a time and determines their order based on the selected attribute. The sorting algorithm module performs the iterative comparisons and rearranges the records accordingly. Finally, a result handler returns the sorted list to the requesting component. Together, these internal parts allow the component to efficiently organize data before it is displayed or used elsewhere in the system.
<u>Dependencies</u>	Sort depends on many components within the Personal Media Tracker application. The search component provides the title name or keyword that creates the list given to the sort component. Additionally, sort is dependent on a database retrieval module. This module provides media titles and information about them to sort from. This function also facilitates other components within the Personal Media Tracking application. A search component provides the list of records and sorting criteria along with a database retrieval module, and the Sort component returns the sorted list after processing.
<u>Interfaces</u>	Sort interacts with other components through defined input and output parameters. The primary inputs include a list of media records and sorting criteria that specifies the attribute used for ordering, such as title or release date. The output of the component is the same list of records arranged in sorted order and returned to the calling component. If invalid parameters are provided or if no data is available to sort, the component may return error messages such as "Invalid sorting parameter" or "No data available to sort." Although the component does not interact directly with the user interface, its results are passed to other modules that display sorted information to the user.
<u>Resources</u>	Sort requires several external system resources to perform its operations. These include CPU processing time to execute the sorting algorithm and primary memory to temporarily store and manipulate the list of records during the sorting process. Temporary buffers may also be used to hold intermediate data while comparisons and swaps are performed. Additionally, the component may rely on system libraries or built-in programming language utilities that support comparison operations or sorting functions. These resources are external to the component but are necessary for it to perform its task efficiently.
<u>Processing</u>	Sort starts when it receives a list of media records and a sorting criterion from another component in the application system like the search component. Initially, the component verifies that the provided list contains valid data and that the sorting parameter is supported. After validation, the component applies a sorting algorithm that compares records according to the specified attribute. The algorithm repeatedly compares records and rearranges them until the list is ordered correctly according to the selected sorting rule. Once finished, the organized list of records is returned to the user through the user interface.
<u>Data</u>	The data used within Sort consists of a temporary list or array of media records provided by another component. The component reorders the temporary list of media titles, matching the user selection from the dropdown menu. Each record contains various attributes about a media title. The initial values of the data are based on the calling component, and the final output contains the same records arranged in a new order according to the selected sorting criterion. The structure and format of these records are defined in the system's data dictionary, which specifies the representation and meaning of each data field.

3.3 TRACKING COMPONENT

<u>Identification</u>	Tracking
<u>Type</u>	Module
<u>Purpose</u>	Tracking allows users to record and save data about a specific media title. This is a core component of the Personal Media Tracker, as it enables users to maintain a personalized list of tracked media.
<u>Functions</u>	Tracking information is retrieved through a form. The form collects information on the media title, viewing progress, personal rating, and review. After submission, the system stores the data and displays it in the user's tracked list. Each record allows the user to modify or update the input after saving.
<u>Subordinates</u>	The tracking modules collect information from the user through input. All data input by the user will validate for any required fields and correct data type. The module will also interact with local storage for writing and retrieving data on tracked titles by associated profiles. These sub-processes ensure accurate and consistent data storage.
<u>Dependencies</u>	This component relies on the associated profiles for specifying which title is tracked per profile. The search component also permits the tracking component to track the title. The local storage system is also required for storing data input by the user.
<u>Interfaces</u>	Interfaces for the tracking component include the user interface that receives input for title tracking. It also communicates with local storage for storing and retrieving tracked titles and the search module interface for selection and tracking of title.
<u>Resources</u>	Tracking communicates with the local storage for storing tracking data. This component also interacts with system memory for data handling and processing power for saving and updating records.
<u>Processing</u>	Tracking begins when the user selects a media title to track. The system displays a tracking form for user input. The user enters tracking information such as viewing progress, rating, and review. The system validates the input and stores the data locally under the active user profile. The tracked list is then updated and displayed. If the user chooses to modify an existing record, the system retrieves the stored data, allows updates through the form, and saves the updated information.
<u>Data</u>	Tracking handles data including media title ID, user profile ID, viewing progress, rating, review, and watch status. This data is stored locally and associated with the active user profile.

3.4 PROFILE COMPONENT

<u>Identification</u>	Profile
<u>Type</u>	Module
<u>Purpose</u>	The Profile module allows users to create and manage user profiles within the Personal Media Tracker. This component enables the system to store and organize tracking data specific to each user.
<u>Functions</u>	The Profile module allows users to create a new profile, select an existing profile, and manage profile data. It stores user-specific information and ensures that all tracked media and associated data are linked to the correct profile.
<u>Subordinates</u>	The Profile module includes profile creation, input validation for profile data, and profile selection logic. It also manages storing and retrieving profile information from local storage.
<u>Dependencies</u>	This component depends on the local storage system to save and retrieve profile data. It also supports other components, such as tracking and review, by providing profile information for stored data.
<u>Interfaces</u>	The profile module interfaces with the user interface for profile creation and selection. It communicates with local storage to save and retrieve profile data and interacts with other modules to provide the active user profile.
<u>Resources</u>	The profile module uses local storage for persistent data storage, system memory for temporary data handling, and processing resources for creating, retrieving, and managing profiles.
<u>Processing</u>	The profile module begins when the user opens the application. The system displays available profiles or prompts the user to create a new profile. The user inputs profile information, which is validated and stored locally. When a profile is selected, the system retrieves the associated data and sets it as the active profile.
<u>Data</u>	The Profile module handles data including user profile ID and associated tracked media data.

4 REUSE AND RELATIONSHIPS TO OTHER PRODUCTS

For teams doing enhancement work, reuse is an important issue. Most enhancement work should focus on extending, rather than replacing, the design and product development from earlier semesters. For teams doing new development, reuse can also be an important strategy. In some cases, there is freeware that could be incorporated. In other cases, there are existing modules or classes that could be adapted. Another possibility is the use of special tools that produce open source results and thus permissible under the terms of this course.

This section should include the following subsections as appropriate:

- How reuse is playing a role in your product design
- How reuse is playing a role in your product implementation (and the motivation for changes)
- If you are not reusing material that is available, then give motivation for why it is being thrown out.

5 DESIGN DECISIONS AND TRADEOFFS

Design decisions were made in order to create a balance between performance, usability, and implementation time. A desktop-based approach was chosen over a web-based application to allow for local data storage and reduce dependency on network connectivity. While a web-based application would allow users to upload or view others' reviews online, the scope of this application is more personalized and private. This approach also simplifies development and improves overall system performance.

Local storage was selected instead of cloud storage to ensure data privacy and reduce reliance on external systems. Cloud storage would allow for data synchronization across multiple devices; however, it would introduce additional complexity such as security concerns, authentication requirements, and increased system overhead.

A modular system architecture was implemented to separate components such as tracking, searching, and sorting. This design simplifies development and allows components to be maintained independently. A more tightly integrated system could reduce communication between components, but it would make future updates and modifications more difficult.

A simple and intuitive user interface was prioritized over a more complex, feature-heavy design to improve usability and reduce the learning curve for users. While a more advanced interface could provide additional customization options, it would require more development time and could negatively impact system performance.

An external database was chosen for media retrieval instead of storing all media locally. This allows users to access a larger and continuously updated dataset. However, this design introduces a dependency on internet connectivity for retrieving media information.

6 PSEUDOCODE FOR COMPONENTS

Utilize the use cases to create pseudocode for components.

7 APPENDICES (IF ANY)

SOFTWARE COMPONENT TEMPLATE FOR SECTION 3

The template given below suggests a reasonable structure for giving a thorough description of each component described in Part 3 of the SDS. The specific information depends in part on the design approach. Your team must adapt this template to your needs and describe it in section 3.1 of your SDS.

<u>Identification</u>	The unique name for the component and the location of the component in the system. Ex: Search
<u>Type</u>	A module, a subprogram, a data file, a control procedure, a class, etc.
<u>Purpose</u>	Function and performance requirements implemented by the design component, including derived requirements. Derived requirements are not explicitly stated in the SRS, but are implied or adjunct to formally stated SDS requirements.
<u>Function</u>	What the component does, the transformation process, the specific inputs that are processed, the algorithms that are used, the outputs that are produced, where the data items are stored, and which data items are modified.
Subordinates	The internal structure of the component, the constituents of the component, and the functional requirements satisfied by each part.
<u>Dependencies</u>	How the component's function and performance relate to other components. How this component is used by other components. The other components that use this component. Interaction details such as timing, interaction conditions (such as order of execution and data sharing), and responsibility for creation, duplication, use, storage, and elimination of components.
<u>Interfaces</u>	Detailed descriptions of all external and internal interfaces as well as of any mechanisms for communicating through messages, parameters, or common data areas. All error messages and error codes should be identified. All screen formats, interactive messages, and other user interface components (originally defined in the SRS) should be given here.
Resources	A complete description of all resources (hardware or software) external to the component but required to carry out its functions. Some examples are CPU execution time, memory (primary, secondary, or archival), buffers, I/O channels, plotters, printers, math libraries, hardware registers, interrupt structures, and system services.
Processing	The full description of the functions presented in the Function subsection. Pseudocode can be used to document algorithms, equations, and logic.
Data	For the data internal to the component, describe the representation method, initial values, use, semantics, and format. This information will probably be recorded in the data dictionary.